

PROGRAMACIÓN DE MICROCONTROLADORES

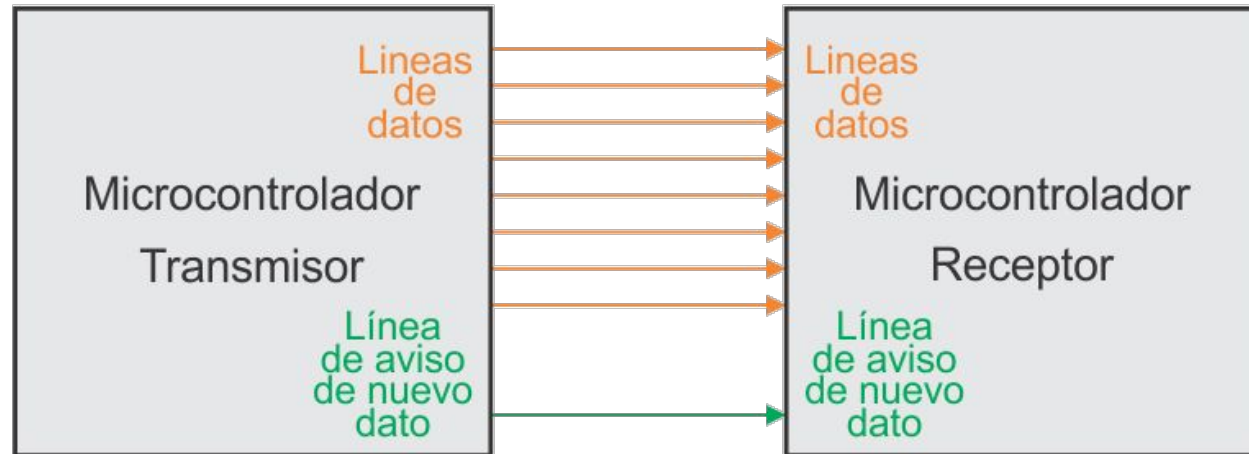
UART

**Carrera de Especialización en Sistemas Embebidos
Facultad de Ingeniería - UBA**

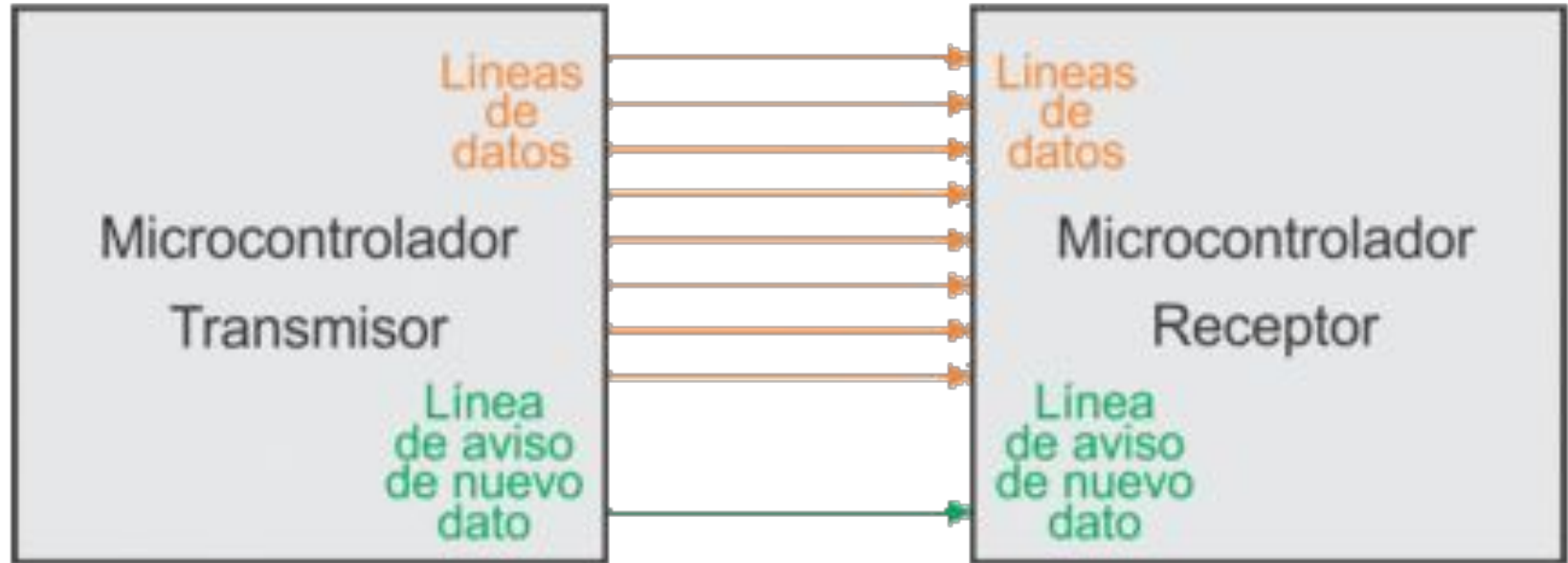
Comunicación entre dispositivos

Se desea enviar datos de 8 bits (1 byte) de un dispositivo a otro.

Una opción es implementar un esquema de **conexión en paralelo** con pines de entrada/salida de propósito general.



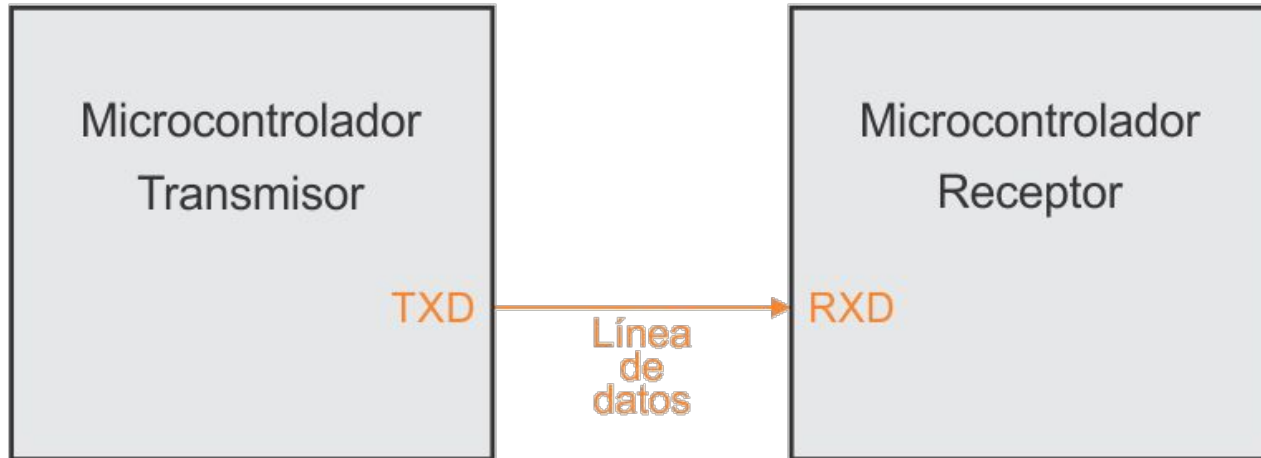
Transmisión de datos en paralelo



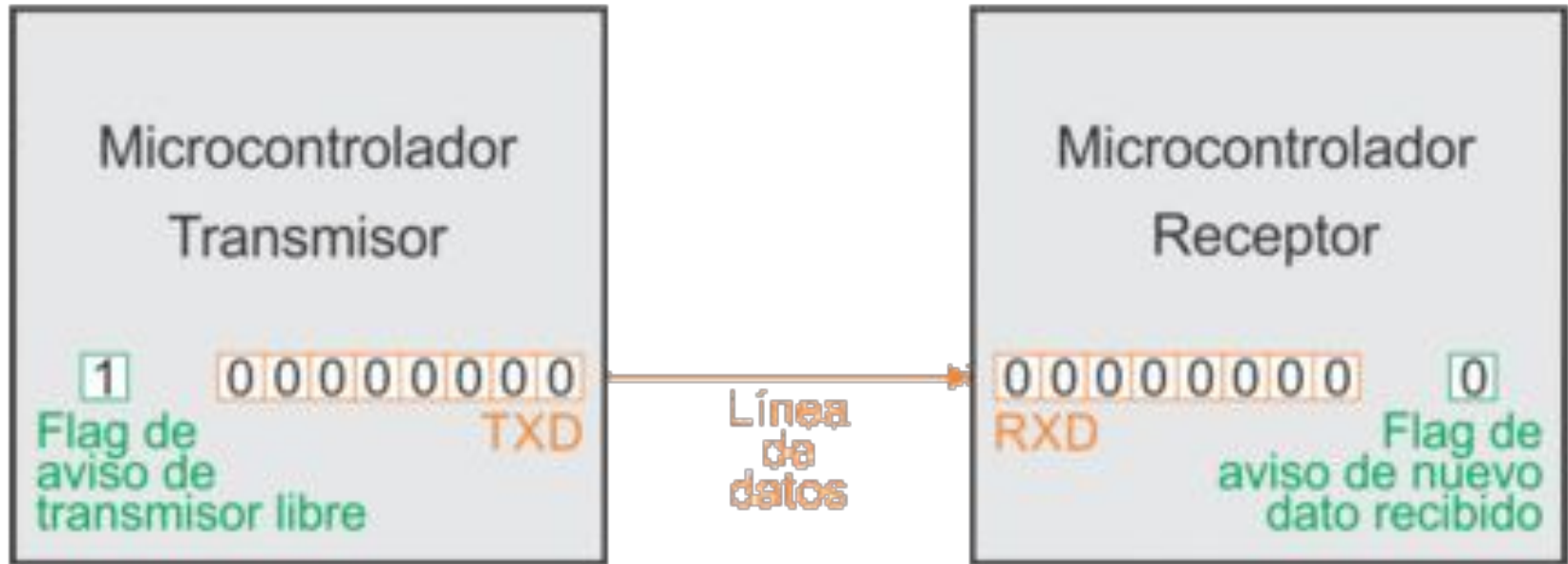
Comunicación entre dispositivos

Un esquema de **conexión en serie** con pines de entrada/salida de propósito general **minimiza la cantidad de conexiones necesarias**.

Un mensaje se va enviar de a un bit a la vez.

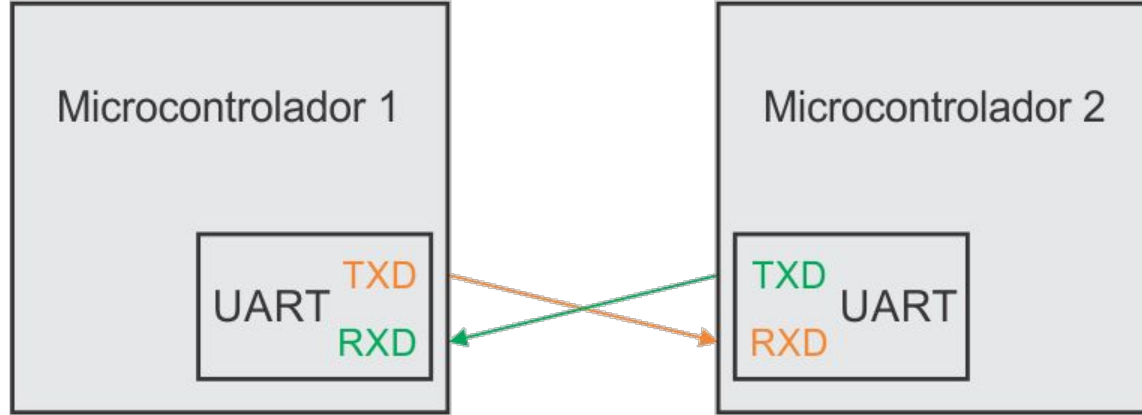


Transmisión de datos en serie



Implementación práctica

En lugar de utilizar pines de entrada y salida de propósito general, se utiliza un periférico llamado **UART** que **permite transmitir y recibir datos en serie entre dispositivos**.



Periférico UART

- UART son las siglas de **Universal Asynchronous Receiver-Transmitter**.
- Se llama **universal** pues posee múltiples características intercambiables, englobando a una amplia gama de sistemas de comunicación.
- Es **asincrónica**, porque no se transmite una señal de clock para sincronizar los datos. Utiliza **bits de start y stop** para sincronizar la comunicación.
- Requiere de **dos líneas** de conexión por dispositivo, una para la **transmisión** (TXD) y otra para la **recepción** (RXD) de datos.
- La comunicación entre dispositivos UART suele ser **full duplex**, es decir, que ambos dispositivos pueden enviar y recibir datos simultáneamente.

Características de la UART

Para establecer una comunicación entre dispositivos se deben pre-acordar las siguientes características:

- Tasa de transmisión de datos.
- Formato de trama del mensaje.
- Estándar de comunicación.

Tasa de transmisión

La tasa de transmisión, o **Baud Rate**, es la cantidad de símbolos por segundo que maneja la comunicación. No se debe confundir con la tasa de bits, llamada bit rate, puesto que

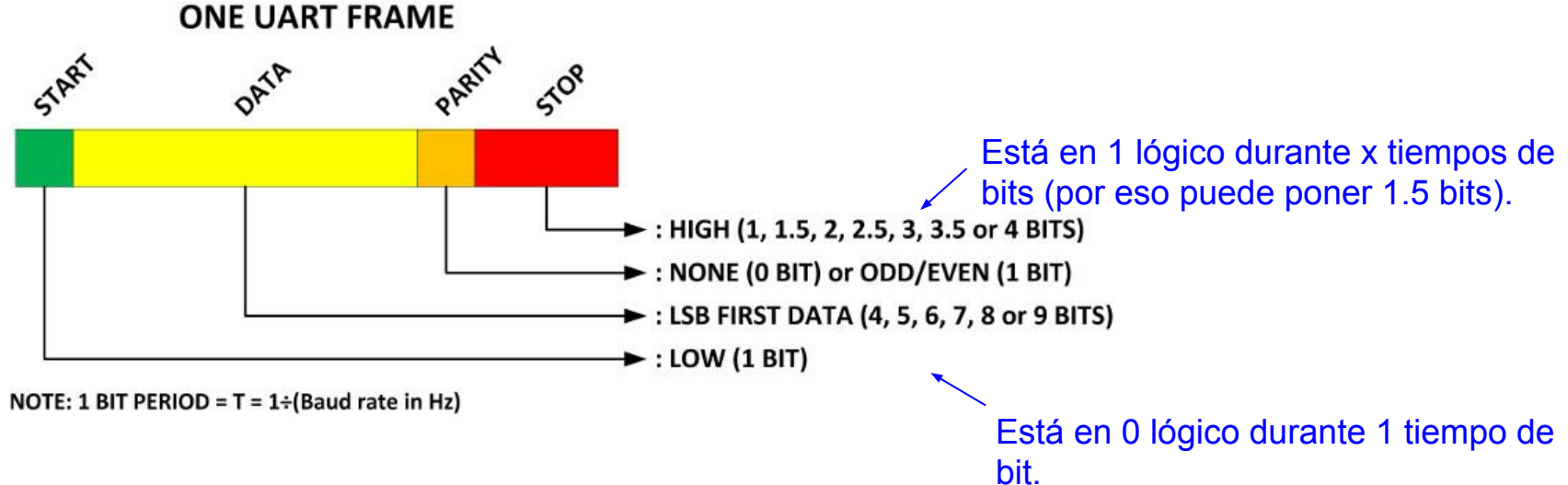
Baud rate = (Bit rate) / (Bits en el símbolo)

Las tasas de transmisión estándar más comunes para la comunicación asincrónica son: **9600 baudios, 38400 baudios, 57600 baudios, 115200 baudios**, aunque existen otras, tanto mayores como menores.

Si bien a simple vista parecería que utilizar una tasa mayor siempre es mejor, las tasas de transferencia entre TX y RX tienen que tener un error menor al 5% para asegurar una comunicación confiable, y es más difícil y caro mantener errores bajos con un Baud Rate alto.

Formato de la trama

La trama especifica cómo están dispuestos los bits dentro de un **paquete** de información.



Formato de la trama

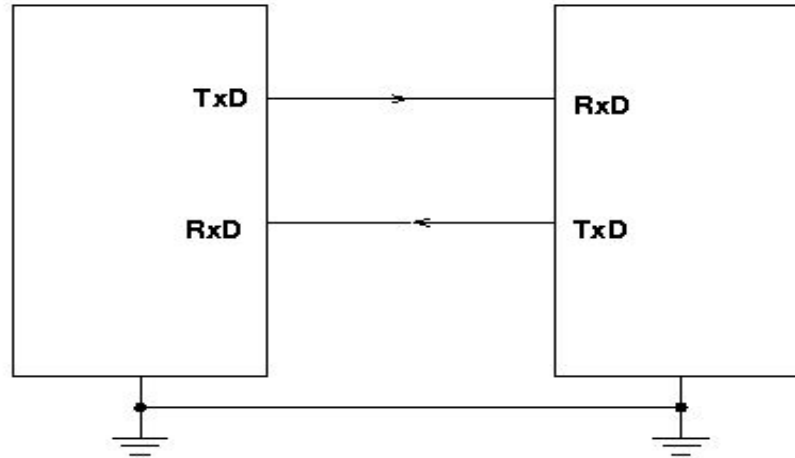
Los **bits de start y stop** sirven para sincronizar la comunicación. Estos bits “extra” no son información útil enviada. Por ejemplo, solo 8 de 10 bits sirven en una configuración típica como 8N1. La información útil dentro de un paquete se llama **carga útil** o **payload**.

El **bit de paridad** sirve para que el destinatario pueda verificar que el mensaje haya sido recibido correctamente.

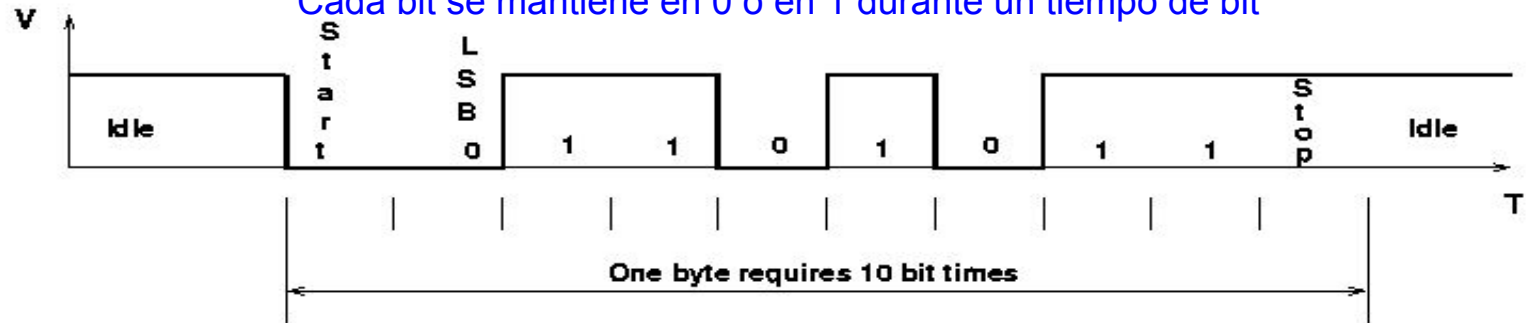
El **transmisor**, al enviar el mensaje, **cuenta la cantidad de unos** en su mensaje e inserta un "1" si la suma es impar y "0" si es par en el campo bit de paridad de la trama.

Luego, el **receptor** vuelve a realizar la suma y chequea este bit para resolver si la trama recibida es correcta.

Asynchronous Serial Communications



Observar que se transmite primero el LSB y por último el MSB
Cada bit se mantiene en 0 o en 1 durante un tiempo de bit



MSB → 0xD6 ← LSB
(bit más significativo) 11010110 (bit menos significativo)

Estándar de comunicación

El periférico UART puede implementar diversos estándares de comunicación definidos por la **Telecommunications Industry Association** (TIA).

Los más utilizados son **RS-232**, **RS-422**, (ambos punto a punto), y **RS-485**, (bus, que permite conectar varios dispositivos).

RS-232 fue el primero creado y es el más simple.

Para conectar una UART del microcontrolador con un puerto serie de una PC se necesita un circuito integrado como el MAX3232 para **adaptar los niveles de tensión**.

En las PC actuales ya es raro encontrar este tipo de periféricos.

Ventajas y desventajas

Ventajas:

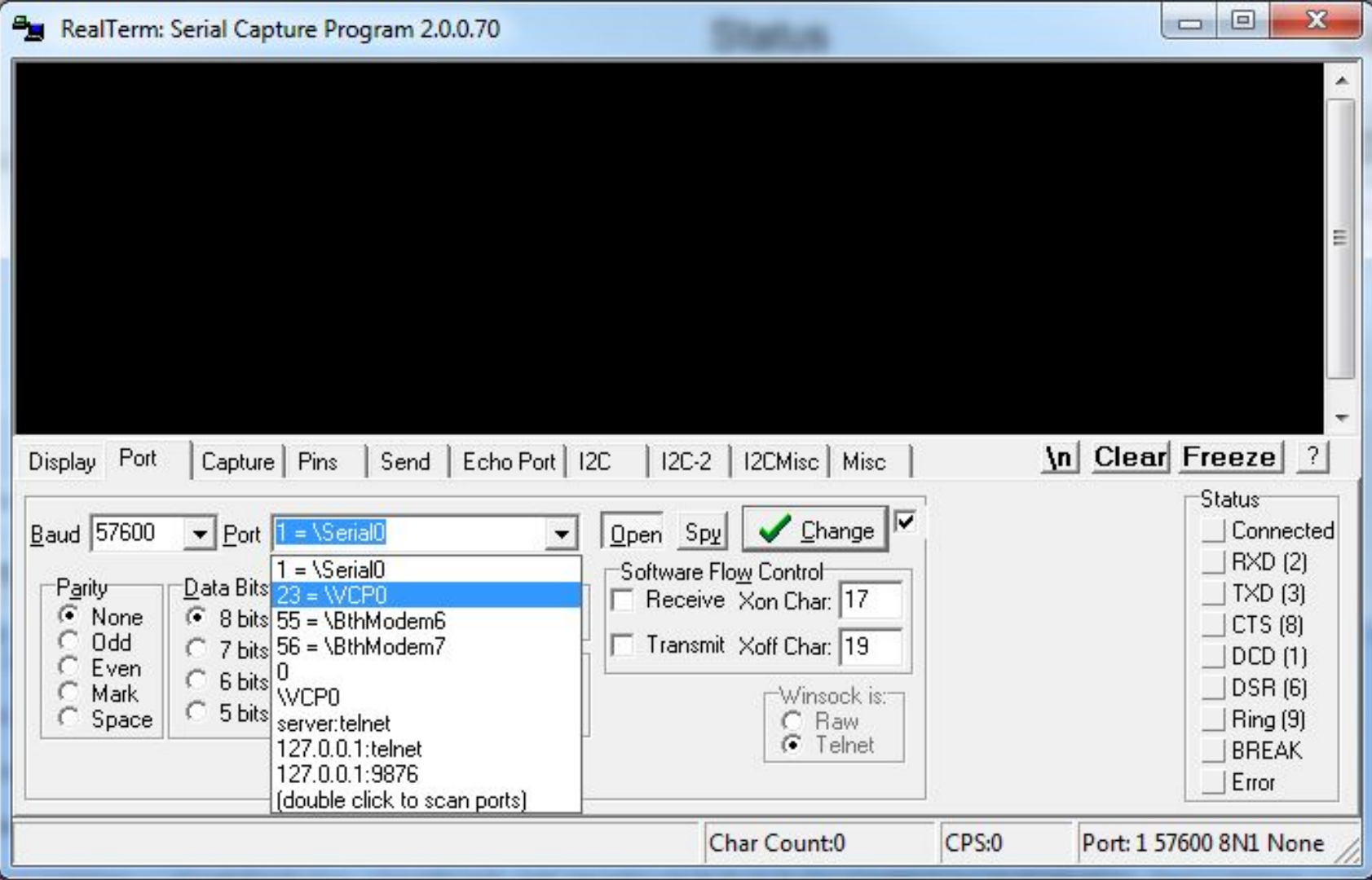
- Es un sistema muy **simple** y difundido y que necesita solo 2 cables.
- Los **circuitos** necesarios para implementar la mayoría de los estándares de comunicaciones son **sencillos y baratos**.

Desventaja:

- Su carácter asincrónico trae acarreado que ambos dispositivos deben **acordar** de antemano la **tasa de transmisión** y utilizar **bits extra** en la trama del mensaje para que la comunicación esté **sincronizada** (START y STOP)

Terminales serie para PC

Emulador de terminal	¿Soporta ANSI?	Sitio oficial / Proyecto
Minicom	 Sí	https://alioth-lists.debian.net/projects/minicom
GNU Screen	 Sí	https://www.gnu.org/software/screen/
PuTTY	 Sí	https://www.putty.org/
Tera Term	 Sí	https://osdn.net/projects/ttssh2/
CoolTerm	 Sí	https://freeware.the-meiers.org/
GtkTerm	 Sí	https://github.com/Jeija/gtkterm
CuteCom	 No	https://cutecom.sourceforge.net/



Display Port Capture Pins Send Echo Port I2C I2C-2 I2CMisc Misc

\n Clear Freeze ?

Baud 57600

Port 1 = \Serial0

Open

Spy

Change



Status

- ☐ Connected
- ☐ RXD (2)
- ☐ TXD (3)
- ☐ CTS (8)
- ☐ DCD (1)
- ☐ DSR (6)
- ☐ Ring (9)
- ☐ BREAK
- ☐ Error

Parity

- ☒ None
- ☐ Odd
- ☐ Even
- ☐ Mark
- ☐ Space

Data Bits

- ☒ 8 bits
- ☐ 7 bits
- ☐ 6 bits
- ☐ 5 bits

1 = \Serial0

23 = \VCP0

55 = \BthModem6

56 = \BthModem7

0

\VCP0

server:telnet

127.0.0.1:telnet

127.0.0.1:9876

(double click to scan ports)

Software Flow Control

☐ Receive

Xon Char: 17

☐ Transmit

Xoff Char: 19

Winsock is:

☐ Raw

☒ Telnet

Char Count:0

CPS:0

Port: 1 57600 8N1 None

Caracteres y strings en C

Un **caracter en lenguaje C** puede almacenarse en **1 byte**.

Para indicar en C un caracter se utilizan comillas rectas simples, por ejemplo,

```
char miCaracter = 'a';
```

guarda en la variable miCaracter el código ASCII de la letra 'a'.

Para lograr un 'enter' se deben utilizar los caracteres '\n' (nueva línea) y '\r' (retorno de carro).

El código ASCII

sigla en inglés de American Standard Code for Information Interchange
(Código Estadounidense Estándar para el Intercambio de Información)

Tomada del sitio:

tecnologiaeinformaticaa.es.tl

Caracteres ASCII de control

00	NULL	(carácter nulo)
01	SOH	(inicio encabezado)
02	STX	(inicio texto)
03	ETX	(fin de texto)
04	EOT	(fin transmisión)
05	ENQ	(consulta)
06	ACK	(reconocimiento)
07	BEL	(timbre)
08	BS	(retroceso)
09	HT	(tab horizontal)
10	LF	(nueva línea)
11	VT	(tab vertical)
12	FF	(nueva página)
13	CR	(retorno de carro)
14	SO	(desplaza afuera)
15	SI	(desplaza adentro)
16	DLE	(esc.vínculo datos)
17	DC1	(control disp. 1)
18	DC2	(control disp. 2)
19	DC3	(control disp. 3)
20	DC4	(control disp. 4)
21	NAK	(conf. negativa)
22	SYN	(inactividad sinc)
23	ETB	(fin bloque trans)
24	CAN	(cancelar)
25	EM	(fin del medio)
26	SUB	(sustitución)
27	ESC	(escape)
28	FS	(sep. archivos)
29	GS	(sep. grupos)
30	RS	(sep. registros)
31	US	(sep. unidades)
127	DEL	(suprimir)

Caracteres ASCII imprimibles

32	espacio	64	@	96	`
33	!	65	A	97	a
34	"	66	B	98	b
35	#	67	C	99	c
36	\$	68	D	100	d
37	%	69	E	101	e
38	&	70	F	102	f
39	'	71	G	103	g
40	(	72	H	104	h
41	)	73	I	105	i
42	*	74	J	106	j
43	+	75	K	107	k
44	,	76	L	108	l
45	-	77	M	109	m
46	.	78	N	110	n
47	/	79	O	111	o
48	0	80	P	112	p
49	1	81	Q	113	q
50	2	82	R	114	r
51	3	83	S	115	s
52	4	84	T	116	t
53	5	85	U	117	u
54	6	86	V	118	v
55	7	87	W	119	w
56	8	88	X	120	x
57	9	89	Y	121	y
58	:	90	Z	122	z
59	;	91	[	123	{
60	<	92	\	124	
61	=	93	]	125	}
62	>	94	^	126	~
63	?	95	_		

ASCII extendido (Página de código 437)

128	Ç	160	á	192	Ł	224	Ó
129	ù	161	í	193	ł	225	ô
130	é	162	ó	194	Ł	226	õ
131	â	163	ú	195	ł	227	ö
132	ä	164	ñ	196	—	228	ø
133	à	165	Ñ	197	+	229	õ
134	ã	166	ª	198	ä	230	µ
135	ç	167	º	199	Ä	231	þ
136	ê	168	¿	200	ℓ	232	ð
137	ë	169	@	201	ƒ	233	ú
138	è	170	¬	202	ℒ	234	Û
139	ï	171	½	203	ƒ	235	Ü
140	î	172	¼	204	ℒ	236	ý
141	ì	173	¡	205	=	237	ÿ
142	Ā	174	«	206	≠	238	ˉ
143	Ă	175	»	207	#	239	˘
144	É	176	⋮	208	ø	240	≡
145	æ	177	⋮	209	ø	241	±
146	Æ	178	⋮	210	Ê	242	—
147	ó	179	 	211	Ê	243	¾
148	ô	180	 	212	È	244	¶
149	ò	181	À	213	Í	245	§
150	ù	182	Ā	214	İ	246	÷
151	û	183	Ă	215	Î	247	˙
152	ÿ	184	⊙	216	Ï	248	˚
153	Ö	185	‖	217	Ɔ	249	…
154	Ü	186	‖	218	┐	250	•
155	ø	187	┐	219	█	251	ˆ
156	£	188	┐	220	█	252	˜
157	ø	189	¢	221	┐	253	²
158	×	190	¥	222	┐	254	■
159	f	191	γ	223	█	255	nbsp

Caracteres y strings en C

Vemos que **en la tabla ASCII que el caracter 'a' es el número 97**, entonces es lo mismo declarar

```
char miCaracter = 'a';
```

que declarar

```
char miCaracter = 97;
```

Luego vemos que todas las letras están en orden y que las mayúsculas están todas a la misma distancia de las minúsculas correspondiente a, por ejemplo,

`'a' - 'A' = 32`

Caracteres y strings en C

De esta forma si queremos **convertir de minúsculas a mayúsculas** debemos **restarle 32**, o sumarle 32 en el caso opuesto, ej:

```
char miCaracter = 'a' - 32 ;
```

queda guardado 'A' en miCaracter

Los números comienzan en el 48, es por eso que **para convertir un número entero de un solo dígito debo sumarle 48**:

```
char miCaracter = 5 + 48;
```

lo que es equivalente a:

```
miCaracter = '5'
```

Caracteres y strings en C

Un **String en C** es simplemente un vector o array de caracteres terminado en el caracter NULL ('\0') o número 0, ej:

```
char myString[5] = { 'H', 'o', 'l', 'a', '\0' };
```

C permite escribir lo anterior como:

```
char myString[ ] = "Hola";
```

String con un enter al final:

```
char myString[ ] = "Hola de nuevo\r\n";
```

Secuencias de escape ANSI

Las **secuencias de escape ANSI** son comandos especiales enviados a través de la terminal para **controlar la apariencia del texto** y la posición del cursor. Son ampliamente utilizadas para:

- Borrar la pantalla o líneas individuales
- Mover el cursor a coordenadas específicas
- Aplicar colores y estilos (negrita, subrayado, etc.)

Estas secuencias comienzan con el carácter **ESC** (`\x1b`) seguido de corchetes y parámetros (`[`), por ejemplo:

`\x1b[2J` # Borra toda la pantalla

`\x1b[10;5H` # Mueve el cursor a fila 10, columna 5



Estándar oficial (ECMA-48):

<https://ecma-international.org/publications-and-standards/standards/ecma-48/>



También conocido como: **ANSI X3.64 o ISO/IEC 6429**

Secuencias de escape ANSI

Para enviar comandos se usa un caracter de escape ESC, por ejemplo:

- 27 → ESC command
- "[2J" → Clear screen command
- 27 → ESC command
- "[H" → Cursor to home command
- Puede ser necesario un retardo de 1 ms para que permita mostrar nuevos mensajes.

```
void Terminal_ClearScreen(void) {  
    const char *clear_cmd = "\x1b[2J\x1b[H"; // Borra pantalla y lleva el cursor al inicio  
    HAL_UART_Transmit(&huart2, (uint8_t*)clear_cmd, strlen(clear_cmd), HAL_MAX_DELAY);  
}
```

STM32F4XX_HAL

La HAL provee dos archivos para abstraerse del periférico, ubicados en
\\drivers\\STM32F4xx_HAL_Driver:

- stm32f4xx_hal_uart.c
- stm32f4xx_hal_uart.h

Las funciones básicas principales son:

- HAL_StatusTypeDef **HAL_UART_Init**(UART_HandleTypeDef *huart)
- HAL_StatusTypeDef **HAL_UART_Transmit**(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);
- HAL_StatusTypeDef **HAL_UART_Receive**(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout);

STM32F4XX_HAL

Para usar la UART con la HAL de ST, lo primero es declarar una variable manejadora del periférico:

```
/* UART handler declaration */
```

```
UART_HandleTypeDef UartHandle;
```

UART_HandleTypeDef es un tipo de dato redefinido en base a una estructura definida en stm32f4xx_hal_uart.h

UART_HandleTypeDef

```
typedef struct {
    USART_TypeDef          *Instance;          /* UART registers base address */
    UART_InitTypeDef       Init;               /* UART communication parameters */
    UART_AdvFeatureInitTypeDef AdvancedInit;   /* UART Advanced Features initialization parameters */

    uint8_t                *pTxBuffPtr;        /* Pointer to UART Tx transfer Buffer */
    uint16_t               TxXferSize;         /* UART Tx Transfer size */
    uint16_t               TxXferCount;        /* UART Tx Transfer Counter */
    uint8_t                *pRxBuffPtr;        /* Pointer to UART Rx transfer Buffer */
    uint16_t               RxXferSize;         /* UART Rx Transfer size */
    uint16_t               RxXferCount;        /* UART Rx Transfer Counter */
    DMA_HandleTypeDef       *hdmatx;           /* UART Tx DMA Handle parameters */
    DMA_HandleTypeDef       *hdmarx;           /* UART Rx DMA Handle parameters */
    HAL_LockTypeDef         Lock;              /* Locking object */
    __IO HAL_UART_StateTypeDef State;          /* UART communication state */
    __IO HAL_UART_ErrorTypeDef ErrorCode;      /* UART Error code */
} UART_HandleTypeDef;
```

```
/*##-1- Configure the UART peripheral #####*/
/* Put the USART peripheral in the Asynchronous mode (UART Mode) */
/* UART configured as follows:
    - Word Length = 8 Bits (7 data bit + 1 parity bit) :
        BE CAREFUL : Program 7 data bits + 1 parity bit in PC HyperTerminal
    - Stop Bit      = One Stop bit
    - Parity        = ODD parity
    - BaudRate      = 9600 baud
    - Hardware flow control disabled (RTS and CTS signals) */
```

```
UartHandle.Instance = USARTx;
```

```
UartHandle.Init.BaudRate    = 9600;
UartHandle.Init.WordLength  = UART_WORDLENGTH_8B;
UartHandle.Init.StopBits    = UART_STOPBITS_1;
UartHandle.Init.Parity      = UART_PARITY_ODD;
UartHandle.Init.HwFlowCtl   = UART_HWCONTROL_NONE;
UartHandle.Init.Mode        = UART_MODE_TX_RX;
UartHandle.Init.OverSampling = UART_OVERSAMPLING_16;
```

```
if (HAL_UART_Init(&UartHandle) != HAL_OK)
{
    /* Initialization Error */
    Error_Handler();
}
```